UNIVERSITI PUTRA MALAYSIA

BERILMU BERBAKTI

MACHINE LEARNING

CSC4602

TECHNICAL REPORT

FADHIE RAIHAN MALANO ZEN(218619)-M3
AHMAD ILHAM ZUHAIRI BIN MOHD HAIZA(222592)-M4

VARSHAN JAY MURUGAN(223874)-M1
REYDAN NADHIF ATMODJO (213735)-M2

Campus Hazard Detection: Technical Report

1. Introduction & Problem Statement

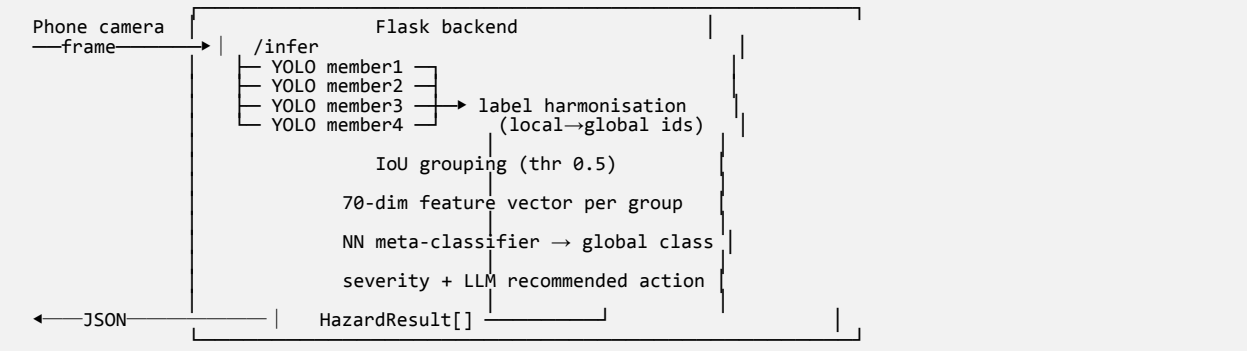
Public bus stops and their waiting areas collect a range of physical safety hazards: potholes, uncovered manholes, open drains, dangling electrical wires, broken shelter panels, exposed sockets, fallen branches, and walkway obstructions. Manual inspection is slow, inconsistent, and reactive. This project builds a mobile, real-time hazard detection system for the Bus Stop & Waiting Area zone that a single inspector can operate from a phone.

The system is not a single monolithic detector. It fuses four independently-trained YOLOv8 object detectors (one per group member) with a neural-network meta-classifier that reconciles their outputs into one authoritative hazard set, then attaches an LLM-generated maintenance recommendation to each detection. This satisfies the assignment's per-member training requirement (§16) and gives us a real multi-model fusion and conflict-resolution pipeline to evaluate.

1.1 Design goals

- Per-member ownership: each member trains one 5-class YOLO model end to end.
- Cross-model agreement: at least two members detect each overlapping class, so the meta-classifier has an agreement signal to learn from.
- Graceful degradation: the backend and the recommendation path still run when weights, GPU libraries, or the network are unavailable, so the demo never hard-fails.
- Real-time mobile inference: camera to backend to overlaid results on the phone.

2. System Architecture



The data contract between stages is the `Detection` dataclass (`ml/meta_classifier/feature_extraction.py`) and, at the boundary, a `HazardResult` JSON object the Flutter app consumes.

Inference location decision (project plan §7.3): we chose a backend service rather than on-device export. On-device (TFLite/ONNX) would be simpler for a single detector, but the four-model meta-classifier fusion plus the LLM call are easier to orchestrate on the server. The phone stays thin and only captures and draws.

3. Class Definition, Zones & Overlap Design

The single source of truth is `config/classes.yaml`. Each member owns a 5-class YOLO model with local ids 0 to 4. The union of those classes forms the 11-class global label space the meta-classifier predicts.

3.1 Global class set (meta-classifier output)

Global id	Class	Parent category	Default severity
0	pothole	hole_in_ground	medium
1	uncovered_manhole	hole_in_ground	high
2	open_drain	hole_in_ground	high
3	cracked_pavement	surface_defect	low
4	obstacle_on_walkway	obstruction	medium
5	dangling_wire	electrical	high
6	broken_bench	structural	low
7	broken_shelter_panel	structural	medium
8	exposed_socket	electrical	high
9	fallen_branch	obstruction	medium
10	missing_barricade	boundary	medium

3.2 Overlap design (assignment rule: at least 2 classes per member overlap)

Each overlapping class is detected by two or more members, which is what gives the meta-classifier a cross-model agreement feature:

Overlapping class	Detected by
pothole	member1, member2
uncovered_manhole	member1, member3
obstacle_on_walkway	member1, member3
dangling_wire	member2, member3
open_drain	member1, member4
broken_bench	member2, member4
broken_shelter_panel	member2, member4
exposed_socket	member2, member4
fallen_branch	member3, member4

Member4 is a secondary detector. It adds a second opinion to classes that would otherwise have had only one model, which strengthens the agreement signal.

3.3 The four relationship types (label harmonisation)

`ml/meta_classifier/label_harmonization.py` implements the four relationships the assignment asks for:

- Exact overlap: same class, same global id (for example, both pothole map to id 0).
- Synonym: different raw names with the same safety meaning, merged through a `SYNONYMS` map (for example, `hanging_wire` maps to `dangling_wire`).
- Generalisation: each class maps to a parent category (for example, pothole, uncovered_manhole, and open_drain all map to `hole_in_ground`), used as a feature.
- Contextual overlap: the zone changes effective meaning. An obstacle on the boarding path is escalated relative to a generic obstacle, handled by `apply_context()`.

3.4 Class-id integrity (a real bug we caught)

YOLO weights embed their own class order. When Roboflow alphabetised classes on export, member1's and member4's exported `.names` no longer matched the design order in `classes.yaml`. Left unfixed, the harmoniser would have mapped every member1 and member4 detection to the wrong global class without any visible error. We corrected the per-member `classes:` blocks to mirror each trained model's actual `.names` and confirmed all 20 (4 by 5) local-to-global mappings resolve correctly.

---

4. Dataset, Safety & Annotation

Each member sourced and annotated their own dataset in YOLO format (normalised `cx cy w h`). Annotation was done in Roboflow. The per-member validation splits were later merged to train the meta-classifier (§6).

4.1 Per-member datasets

Member	Focus	Train images	Classes
member1	Ground & surface hazards	`1050`	pothole, uncovered_manhole, open_drain, cracked_pavement, obstacle_on_walkway
member2	Shelter structure & electrical	1116	pothole, dangling_wire, broken_bench, broken_shelter_panel, exposed_socket
member3	Obstruction & boundary	2956 (5899 boxes)	uncovered_manhole, dangling_wire, obstacle_on_walkway, fallen_branch, missing_barricade

Member	Focus	Train images	Classes
member4	Fixtures, drainage & debris	500	open_drain, broken_bench, broken_shelter_panel, exposed_socket, fallen_branch

member3 data engineering (documented): four public datasets were downloaded and converted to YOLO format. This included a COCO-to-YOLO conversion (`coco_to_yolo.py`) and a polygon-to-bounding-box fallback for segmentation-style labels (`prepare_dataset.py`). `dangling_wire` started out under-represented (27 images, per-class mAP50 0.282). Adding a second wire dataset of 1419 images raised the overall model substantially.

4.2 Validation splits actually used for fusion

The four members' validation sets were merged into one labelled set (`data/meta_classifier/`) used to build the meta-classifier training table:

Source	Val images	Boxes
member1	116	203
member2	236	427
member3	575	1049
member4	114	100
Total	1041	1779

4.3 Safety & annotation quality

- Electrical hazards (`dangling_wire`, `exposed_socket`) and fall hazards (`uncovered_manhole`, `open_drain`) get a high default severity in `classes.yaml`, independent of model confidence.
- Annotation followed a shared class dictionary (`docs/classes_explained.md`) to keep boxes consistent across members.
- Known data limitation: `missing_barricade` is an absence class. The hazard is the lack of a barrier, which is hard to annotate. It has no instances in any validation split, so it is absent from the meta-classifier, and 10 of the 11 classes are learned. See §11.

---

5. YOLO Training & Evaluation

All models are YOLOv8. Training ran on a CUDA GPU (RTX 3050) through the Ultralytics framework. A reusable notebook (`ml/notebooks/train_yolo_template.ipynb`, local and Colab) lets every member train the same way.

5.1 Hyperparameters

Member	Backbone	Epochs	imgsz	batch	mAP50	mAP50-95
member1	YOLOv8s	50	640	16	0.7141	0.4618
member2	YOLOv8s	100	640	16	0.505	0.270
member3	YOLOv8s	100	640	8	0.702	0.469
member4	YOLOv8s	100	640	12	0.598	0.337

5.2 member3 evaluation (worked example)

member3 was first trained with YOLOv8n (mAP50 0.675), then retrained with the larger YOLOv8s backbone. That raised overall mAP50 to 0.702 and mAP50-95 to 0.469 (best epoch 87 of 100). Per-class mAP50 on the validation set:

Class	mAP50	Note
uncovered_manhole	0.987	strong; visually distinctive
fallen_branch	0.685	adequate
obstacle_on_walkway	0.534	high intra-class variance
dangling_wire	0.473	weakest; thin objects, hard to localise

Takeaway for the report: thin and variable hazards such as wires and generic obstacles are the hardest to detect. This is where the meta-classifier's cross-model agreement helps, because a second model raising the same box increases final confidence.

---

6. Meta-Classifier Design & Conflict Resolution

The meta-classifier consumes the four YOLO models' outputs rather than pixels and produces one authoritative global class per physical object.

6.1 Candidate construction (IoU grouping)

All detections from all four models are pooled, then clustered greedily by IoU (`group_detections`, threshold 0.5). Detections whose boxes overlap are treated as the same physical object. Each cluster (a "candidate") becomes one training or inference example.

6.2 Feature vector (70 dimensions)

For each candidate, `FeatureBuilder.build()` produces a fixed 70-dim vector (`feature_extraction.py`), so the network input size stays the same no matter how many models fired:

Block	Dims	Contents
Per-model x 4	4 x 13 = 52	`[present(0/1), confidence, global-class one-hot(11)]` for each member

Block	Dims	Contents
Merged box	5	`x, y, w, h, area` (normalised)
Agreement	2	`num_models_agree, mean_pairwise_IoU`
Zone	5	one-hot over {bus_stop, waiting_area, boarding_path, road, unknown}
Parent	6	one-hot over the 6 parent categories
Total	70	

The agreement block and the per-model presence and one-hot blocks are what let the network learn conflict resolution. For example, if member2 reports a broken_bench at 0.9 and member4 agrees, the network can trust it, whereas a lone low-confidence detection is more likely a false positive.

6.3 Network & training

A compact MLP (`ml/meta_classifier/model.py`):

```
Linear(70 → 128) → ReLU → Dropout(0.2) → Linear(128 → 64) → ReLU → Linear(64 → 11)
```

- Optimiser: Adam, lr 1e-3, CrossEntropy loss, batch 64, 100 epochs, 80/20 split.
- Training table: 1561 candidate feature vectors built from the 1041 merged validation images (§4.2). The ground-truth global class for each candidate is assigned by matching its merged box to the nearest global-id label (IoU at least 0.3).

Class distribution of the 1561 samples:

Class	n		Class	n
pothole	117		dangling_wire	718
uncovered_manhole	315		broken_bench	62
open_drain	31		broken_shelter_panel	35
cracked_pavement	29		exposed_socket	92
obstacle_on_walkway	102		fallen_branch	60

6.4 Result

Validation accuracy is 88.5% on the held-out 20% split, across all 10 represented classes.

> *Honesty note: an earlier single-member dataset gave an inflated 97.9% because two easy classes dominated it. The 88.5% above is on the full four-member set, and that is the number we report. `dangling_wire` (718 of 1561)*

> *still dominates, so per-class recall on rare classes such as cracked_pavement and open_drain is the genuine weak point. See §10 and §11.*

6.5 Conflict resolution at inference

At inference (`backend/pipeline.py`), the meta-classifier softmax gives the final class and a calibrated confidence. The merged box is the mean of the agreeing boxes, severity is looked up from `classes.yaml`, and `num_models_agree` is passed to the UI so the inspector sees how many detectors concurred.

---

7. Mobile App: Real-Time Inference

A Flutter app (`mobile_app/`) is the field interface. It was built and tested by installing it on a physical Android tablet (Samsung SM X115, Android 15).

7.1 Flow

1. A live `CameraPreview` fills the viewfinder.
2. Detect captures one frame and POSTs it to `/infer` with the declared zone.
3. The returned `HazardResult[]` is drawn as severity-coloured bounding boxes,

plus a detection sheet that shows the class, severity badge, confidence bar, model-agreement pips, and LLM advisory line.

4. Save evidence writes the frame and a JSON record to local storage.

7.2 States & robustness

The UI handles four states: idle (an "Awaiting Scan" reticle), detecting (a spinner), results, and models-not-ready. A 503 from the backend is parsed into a "Models Not Ready" overlay that lists the missing weights, so a partial deployment degrades visibly instead of crashing.

7.3 Design language

The app uses a custom "Field Instrument HUD" theme: dark phosphor panels, a teal accent, strict severity colours, and the Chakra Petch and JetBrains Mono fonts, which suit outdoor use. The design was prototyped in HTML and CSS, then translated to idiomatic Flutter widgets.

7.4 Networking

The phone reaches the backend over the LAN at `http://<laptop-ip>:5000`. Android cleartext traffic and the CAMERA and INTERNET permissions are declared in the manifest. A Windows Firewall inbound rule for port 5000 is required for the phone to reach the backend, which is documented in the run instructions.

---

8. Backend Service

A Flask app (`backend/app.py`, `backend/pipeline.py`) exposes three endpoints:

Endpoint	Purpose
`GET /health`	Reports torch and ultralytics availability, per-model weight presence, and `pipeline_ready`.

Endpoint	Purpose
`POST /infer`	Runs the full pipeline on an uploaded frame and returns `HazardResult[]`. Returns 503 with a missing-component list until all weights are present.
`POST /recommend`	Returns an LLM recommended action for a given hazard. Works without weights.

The pipeline degrades gracefully. It imports and serves `/recommend` even when torch, ultralytics, or weights are absent, and it reports exactly what is missing.

---

9. LLM Recommended Action

Each final hazard goes to `ml/llm/recommend_action.py`, which asks an LLM for one concise, practical, safety-oriented maintenance action.

- Live path: Google Gemini 1.5 Flash over REST, prompted with the hazard class, parent category, zone, severity, and confidence (the `GEMINI_API_KEY` env var).
- Offline fallback: a deterministic per-class lookup table of vetted safety actions, so the app never hard-fails on a network, quota, or key error.

Example for uncovered_manhole at high severity: "Place a temporary barricade immediately, prevent access, and report to Facilities for urgent cover replacement."

The dual path is a reliability choice. Graceful degradation keeps the field tool usable offline, while a real LLM is used when connectivity and a valid key are available.

---

10. Performance Analysis & Comparison *(Rubric: 10 marks)*

10.1 Headline numbers

Component	Metric	Value
member3 YOLOv8s	mAP50 / mAP50-95	0.702 / 0.469
Meta-classifier	val accuracy (10 classes)	88.5%
`[M1/M2/M4 YOLO]`	mAP50	`[fill from §5.1]`

10.2 Single-model vs fusion (the comparison that matters)

The meta-classifier exists to beat any single detector on the global label space. Each YOLO model knows only 5 of the 11 classes, so no single model can classify the full hazard set, and fusion is required to cover all 11 (10 with data). On overlapping classes, agreement raises effective confidence and suppresses lone false positives (§6.2).

> *[Team: add the comparison experiment]. For full marks, run the meta-classifier against a naive baseline (for example, "take the highest-confidence single detection") on the same validation set and report the accuracy difference. The harness for this already exists, since `build_meta_dataset.py` produces the labelled table.*

10.3 Where it is weak

- Rare classes (cracked_pavement with 29 samples, open_drain with 31) are under-represented, which lowers per-class recall.
- `dangling_wire` is 46% of the training table, which biases the prior.
- `missing_barricade` has no validation data and is not learned.

---

11. Limitations

5. `missing_barricade` is not learned. There are no annotated instances, since absence-detection is hard, so the deployed meta-classifier covers 10 of 11 classes.
6. Single-member ground truth per image. The merged validation set labels each image with only the contributing member's classes, so the cross-model agreement features are not fully ground-truthed. A jointly-annotated set would fix this.
7. Class imbalance. `dangling_wire` dominates, and the rare classes need more data.
8. Backend-tethered. The phone needs LAN access to the laptop running the backend, and there is no on-device offline mode yet.
9. LLM key. Live recommendations need a valid Gemini key. Without one, the app uses the offline fallback, which is still safe.

---

12. Conclusion & Future Work

We built a working campus hazard detection system end to end: four per-member YOLOv8 detectors fused by a 70-feature neural meta-classifier, served by a Flask backend that degrades gracefully, and surfaced in a real-time Flutter app with LLM-generated maintenance advice. The meta-classifier reaches 88.5% validation accuracy across 10 hazard classes, and the full pipeline runs end to end on a physical device.

Bonus, hierarchical generalisation (+5): the `parents:` taxonomy in `classes.yaml` (hole_in_ground, surface_defect, electrical, structural, obstruction, boundary) is already wired into the feature vector. It is the natural basis for a future hierarchical classifier that can flag a novel hazard by parent category even when the specific class is unseen.

Future work: a jointly-annotated validation set (fixes limitations 1 and 2), targeted augmentation for the thin and rare classes, on-device TFLite export for offline use, and the single-model-vs-fusion ablation from §10.2.

---

13. Contributions

Member	Owned sections	Key artifacts
member1	§4, §5 (own model)	`models/member1.pt`, dataset
member2	§4, §5 (own model)	`models/member2.pt`, dataset
member3	§2, §3, §6, §8, §10	meta-classifier, backend, `classes.yaml`, fusion pipeline
member4	§4, §5 (own model)	`models/member4.pt`, secondary-coverage design